ClassRename

dex 문제 발생할 수 있음

외부 패키지 제외하기?

```
ClassRename 난독화 클래스
-----------------------------------
목적:
- Smali 코드 내 클래스 이름 난독화.
- Android Manifest 및 XML 파일에서 클래스 참조도 일괄 변경.
- 역공학과 정적 분석 난이도 향상.

동작 방식:
   1. Manifest 파일에서 패키지명 추출 및 암호화.
   2. APK 내 모든 Smali 파일에서 클래스 선언 탐색.
      - 내부 클래스(.annotation, InnerClass)도 처리.
      - 숫자나 R 클래스는 제외.
      - 문자열 및 슬래시($, /) 표기 변환 처리.
   3. rename_transformations 매핑 생성.
   4. Smali 코드 내 클래스 사용 부분 문자열 및 참조 수정.
   5. res/xml, layout 폴더 내 XML 파일에서 클래스명 변경.
   6. Manifest, XML, Smali 파일 모두 일관되게 변경.

특징:
   - 클래스명 MD5 기반 난독화.
   - 패키지명 암호화 및 sharedUserId 난독화.
   - ignore_package_names로 난독화 제외 패키지 지정 가능.
   - 멀티 레벨 내부 클래스 처리 가능.

주의 사항:
   - Manifest에서 패키지명 추출 실패 시 예외 발생.
   - Smali, XML, Manifest 파일 모두 정확히 반영해야 앱 정상 동작 가능.
   - R 클래스 및 숫자 기반 클래스명은 난독화 제외.
"""
```

테스트 앱에 대한 이유(선정 이유,주의 할 점)를 통한 고도화 방향 잡기

선정 이유:

1. 복잡한 구조: 멀티 모듈, DI(Hilt), Compose, Coroutine 등 모듈화 및 여러 오픈소스, 최신 기술이 탑재되어 있음

2. 네트워크·DB·로컬 저장소 존재: 코드/리소스/네트워크/데이터 계층 난독화 테스트 가능

3. Compose 코드 난독화 영향 확인 가능: Obfuscapk이 smali 수준에서 처리하므로 Compose-generated 코드 난독화 결과 분석 가능

4. 실제 앱처럼 작동: 완성도가 높은 오픈소스앱이므로 난독화 후 동작 비교가 쉬움

주의 할 점

멀티덱스 & Hilt 관련 이슈

• Hilt가 내부적으로 reflection을 사용하기 때문에, Obfuscapk의 Rename 플러그인이 잘못 동작하면

ClassNotFoundException 발생 가능.

Proto DataStore / Serialization

• 자동 생성된 클래스 이름이 바뀌면 데이터 직렬화 실패 가능

Gradle KTS 기반이라 빌드·서명 스크립트 커스터마이징 필요

•Obfuscapk는 apk 완성본 기준으로 동작하므로, Now in Android을 먼저 빌드 → release APK 추출 → Obfuscapk 적용 → 재서명.

• Rebuild – ReSign의 빌드-사이닝 과정을 준수

개념 상세 설명

1. DI(Dependency injection) & Hilt

개념

- **DI (의존성 주입):** 요리를 할 때(클래스 A가 동작할 때), 재료(클래스 B)를 직접 사러 가는 게 아니라, **누군가 문 앞에 배달해 주는 것**
- **Hilt:** 이 배달 시스템을 자동으로 관리해 주는 배달 업체(라이브러리)
- **Reflection (리플렉션):** 배달 기사가 주문서를 보고 "김철수 고객님 댁이 어디지?" 하고 **이름표를 보고 찾아가는 행위**

난독화 시 문제점

Hilt는 @Inject 같은 스티커가 붙은 클래스를 찾아서 배달해준다.
이때 내부적으로 "com.example.UserViewModel"이라는 이름표(문자열)**를 보고 찾는다.

- **원본:** Hilt가 `"com.example.UserViewModel"`을 찾음
- **Class Rename 적용 후:** `UserViewModel` 클래스 이름이 `a.b.C`로 바뀜.
- **결과:** Hilt는 여전히 `"com.example.UserViewModel"`을 찾으려고 시도하다가 `ClassNotFoundException` 오류를 내며 앱이 죽는다.

고도화 포인트: Hilt가 사용하는 클래스(주로 `ViewModel`, `Module` 등)는 이름을 바꾸지 않도록 **예외 처리(Keep Rules)** 해야 한다.

---

2. Serialization(직렬화) & Proto DataStore

개념 (비유: 가구 포장)

- **Serialization (직렬화):** 완성된 가구(객체)를 택배로 보내기 위해 분해해서 상자에 넣는 과정(JSON, XML, Byte)
- **Deserialization (역직렬화):** 상자에서 꺼내서 다시 가구로 **조립하는 과정**
- **Proto DataStore:** 데이터를 아주 작고 빠르게 저장하기 위해, `protobuf`라는 특수한 설계도를 이용해 직렬화하는 최신 저장소

난독화 시 문제점

원본 코드

```
data class User(val name: String, val age: Int)
```

```
// 저장된 데이터: {"name": "binseoyun", "age": 1}
```

Class Rename 적용 후

```
data class a(val b: String, val c: Int) // name → b, age → c 로 바뀜
```

- **결과:** 저장소에서 `{"name": "binseoyun"}` 를 꺼내와서 조립하려는데, 바뀐 클래스 `a` 에는 `name` 이 없고 `b`. ⇒ **조립 실패(Crash)하거나 데이터가 날아간다**

> 고도화 포인트: @Serializable 같은 어노테이션이 붙은 데이터 클래스는 이름과 필드명을 절대 바꾸면 안된다

---

3. Jetpack Compose

난독화 시 문제점
Compose는 코드가 곧 UI
컴파일러가 코드를 아주 복잡하게 변환하는데, 이 과정에서 **특정 함수 이름이나 구조**에 의존하는 경우가 많다.
Obfuscapk가 Smali 수준에서 함수 이름을 막 바꿔버리면, Compose가 내부적으로 "어? 아까 그 그리기 함수 어디 갔지?" 하고 길을 잃을 수 있습니다.

> 고도화 포인트: Compose 관련 함수들(주로 @Composable 어노테이션이 붙은 함수)은 난독화 시 매우 신중해야 하거나, 제외해야 한다

---

4. Multidex(멀티 덱스)

개념

- 안드로이드 앱(APK)은 내부에 `classes.dex` 라는 파일에 코드를 담는다
- 그런데 이 책 한 권에 담을 수 있는 페이지 수(메서드 개수)가 **65,536개**로 제한되어 있다
- "Now in Android"처럼 큰 앱은 이 제한을 넘어가기 때문에, `classes.dex` , `classes2.dex` , `classes3.dex` **...** 처럼 책을 여러 권으로 쪼갠다 ⇒ 이것이 **멀티덱스**

난독화 시 문제점

- Obfuscapk가 `classes.dex` 만 읽어서 난독화하면 어떻게 될까?
- (`classes2.dex`)에 있는 코드는 난독화가 안 된다
- 1권에 있는 코드가 이름을 `a` 로 바꿨는데, 2권에 있는 코드는 여전히 원래 이름인 `User` 를 부르고 있으면 **연결이 끊겨서 앱이 죽을 수도 있다.**

> 고도화 포인트: classes_rename 플러그인은 APK 안에 있는 모든 dex 파일을 다 읽어서, 통합적으로 이름을 관리하고 바꿔야 한다.

---

5. Gradle KTS

개념
• 앱을 빌드(컴파일 + 포장)하는 도구인 **Gradle**의 설정 파일

- `Kotlin` (`KTS`) 언어로 작성

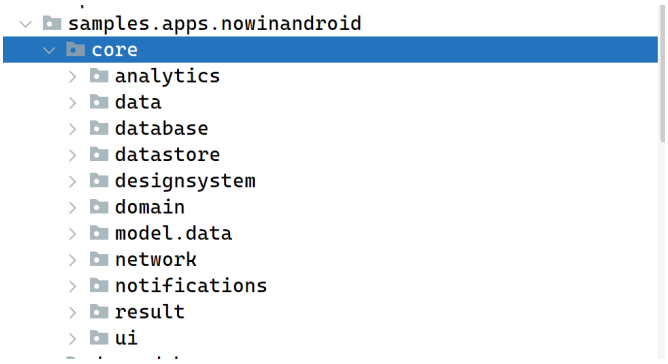
난독화 시 문제점

.
• **"빌드 ⇒ 난독화⇒ 재서명"** 순서를 이해하고 스크립트로 자동화할 수 있어야 한다.

---

6. 네트워크,DB,로컬 저장소 존재

jadx를 통해 apk 파일을 열었을 때 core 패키지 하단의 구조



패키지 상세 설명

패키지 이름	역할 (Role)	비유	난독화 위험도	이유
`core/network`	**네트워크 통신**	**무역항**	**최고**	서버에서 오는 데이터(`DTO`)가 정의된 곳. `@Serializable` 이 잔뜩 있음. 이름 바꾸면 통신 두절.
`core/datastore`	**간편 저장소**	**사물함**	**최고**	앱 설정 등을 저장. Proto Buffers를 쓰기 때문에 구조가 바뀌면 저장된 걸 못 읽음.
`core/database`	**로컬 DB (Room)**	**창고**	**높음**	SQLite DB 테이블과 매핑됨. 클래스 이름이 곧 테이블 이름이 될 수 있음.
`core/model`	**도메인 모델**	**가공품**	**중간**	앱 내부에서 쓰는 데이터 객체. 직접 통신엔 안 쓰이지만, `Parcelable` 등으로 화면 간 전달될 때 이름이 중요할 수 있음.
`core/data`	**데이터 저장소 (Repo)**	**물류센터**	**중간**	Network와 Database에서 데이터를 가져와 정리함. Hilt(`@Inject`)가 많이 쓰임.
`core/domain`	**비즈니스 로직**	**두뇌**	**중간**	"뉴스를 가져와라" 같은 명령을 내림. Hilt 의존성이 강함.
`feature/...`	**화면(UI)**	**매장**	**중간**	`ViewModel` (`@HiltViewModel`)이 존재함. 이름 바뀌면 주입 실패.

⇒ core/network , core/datastore, core/database 세곳의 클래스 이름을 바꾸면 외부와의 소통이 단절됨

⇒ 해당 내부의 클래스는(특히 어노테이션이 붙은 것들)아예 난독화 대상에서 제외

요약 : class_rename 고도화를 위해 해야 할 일

이름을 바꾸면 안되는 것들을 얼마나 잘 똑똑하게 찾아내서 난독화를 진행하게 할 것인지

- **Hilt:** `@HiltAndroidApp` , `@Inject` 등이 붙은 클래스는 건드리면 안됨
- **DataStore:** `@Serializable` 붙은 애들은 건드리면 안됨
- **Compose:** `@Composable` 함수 이름은 조심해야 함
- **Multidex:** `classes.dex` 만 처리하는 게 아니라 `classes2.dex` 등 .. 모든 dex 파일을 한꺼번에 처리해야 함

⇒ Smail 코드에서 저 어노테이션(@)들을 어떻게 찾아낼 것인가가 관건

⇒ Keep Rules 를 만들어서 클래스의 이름을 바꾸지 않고 건너뛸 규칙을 만들어야 함

class_rename 고도화 논리

- **스캔:** Smali 파일을 읽는다.
- **탐지:** 파일 내용 중에 (`@Serializable` 등)가 있는지 본다.
- **판단:** 하나라도 있으면? -> **"이 파일은 건너뛰어(Skip/Keep)!"**
- **실행:** 지뢰가 없는 순수 로직 클래스만 이름을 `a` , `b` , `c` 로 바꾼다.

apk 파일 분석(어노테이션 중심, 패키지 구조)를 통한 고도화 진행

@HiltAndroidApp 어노테이션

- 안드로이드 앱 시작점이 클래스에 붙어서, 앱 전역에 사용할 DI를 초기화
- 종속 항목 삽입을 할 수 있는 애플리케이션의 기본 클래스가 포함된 Hilt 생성
- Keep Rule에 무조건 포함

keep Rule에 포함되어야 하는 이유

1. AndroidManifest.xml 에
   `android:name="com.google.samples.apps.nowinandroid.NiaApplication"` 라고 명시됨 ⇒
   해당 클래스의 이름을 난독화 시켜버리면 앱 실행 조차 안됨 ⇒ 무조건 keep rule에 포함하는 고도화 진행
2. Hilt는 이 어노테이션을 보고 `Hilt_NiaApplication`이라는 부모 클래스를 자동으로 만든다. 이름이 바뀌면 이 연결고리가 끊어진다.

<아래는 @HiltAnroidApp 어노테이션이 선언된 클래스>

```java
package com.google.samples.apps.nowinandroid;

import coil.ImageLoader;
import coil.ImageLoaderFactory;
import com.google.samples.apps.nowinandroid.sync.initializers.Sync;
import com.google.samples.apps.nowinandroid.util.ProfileVerifierLogger;
import dagger.Lazy;
import dagger.hilt.android.HiltAndroidApp;
import javax.inject.Inject;
import kotlin.Metadata;
import kotlin.jvm.internal.Intrinsics;

/* compiled from: NiaApplication.kt */
@Metadata(d1 = {"\u0000,\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\b\u0003\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0
@HiltAndroidApp
/* loaded from: classes5.dex */
public final class NiaApplication extends Hilt_NiaApplication implements ImageLoaderFactory {
    public static final int $stable;

    @Inject
    public Lazy<ImageLoader> imageLoader;

    @Inject
    public ProfileVerifierLogger profileVerifierLogger;

    static {
        boolean[] zArr = (boolean[]) NiaApplication$$ExternalSynthetic$Condy0.get();
        $stable = 8;
        zArr[12] = true;
    }

    public NiaApplication() {
        ((boolean[]) NiaApplication$$ExternalSynthetic$Condy0.get())[0] = true;
    }

    public final Lazy<ImageLoader> getImageLoader() {
        boolean[] zArr = (boolean[]) NiaApplication$$ExternalSynthetic$Condy0.get();
        Lazy<ImageLoader> lazy = this.imageLoader;
        if (lazy != null) {
            zArr[1] = true;
            return lazy;
        }
        Intrinsics.throwUninitializedPropertyAccessException("imageLoader");
        zArr[2] = true;
```

<AndroidManifest.xml 에서
`android:name="com.google.samples.apps.nowinandroid.NiaApplication"` 확인 가능 >

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionCode="8"
    android:versionName="0.1.2"
    android:compileSdkVersion="34"
    android:compileSdkVersionCodename="14"
    package="com.google.samples.apps.nowinandroid.demo.debug"
    platformBuildVersionCode="34"
    platformBuildVersionName="14">
    <uses-sdk
        android:minSdkVersion="21"
        android:targetSdkVersion="34"/>
    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
    <uses-permission android:name="android.permission.WAKE_LOCK"/>
    <uses-permission android:name="com.google.android.finsky.permission.BIND_GET_INSTALL_REFERRER_SERVICE"/>
    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>
    <permission
        android:name="com.google.samples.apps.nowinandroid.demo.debug.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION"
        android:protectionLevel="signature"/>
    <uses-permission android:name="com.google.samples.apps.nowinandroid.demo.debug.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION"/>
    <application
        android:theme="@style/Theme.Nia.Splash"
        android:label="@string/app_name"
        android:icon="@mipmap/ic_launcher"
        android:name="com.google.samples.apps.nowinandroid.NiaApplication"
        android:debuggable="true"
        android:allowBackup="true"
        android:supportsRtl="true"
        android:extractNativeLibs="true"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:appComponentFactory="androidx.core.app.CoreComponentFactory"
        android:enableOnBackInvokedCallback="true">
        <profileable android:shell="true"/>
        <activity
```

smail 파일에서 HiltAndroidApp 어노테이션 탐색 방법

1. .smail 파일을 연다
2. .annotation runtime Ldagger/hilt/android/HiltAndroidApp; 가 있는지 확인한다
3. .class ... Lcom/... 패턴의 정규식을 찾아서 이름을 추출한다
   (com 폴더 하위에 존재하니)

<NiaApplication.smali 예시>

```
# [구역 1] 클래스 정의부 (Class Definition)
# 파일의 맨 윗부분에 있습니다. 이 파일이 '누구'인지 정의
.class public final Lcom/google/samples/apps/nowinandroid/NiaApplication;
.super Landroid/app/Application;
.source "NiaApplication.kt"


# [구역 2] 어노테이션 정의부 (Annotation Block)
# "@HiltAndroidApp"

# "런타임(앱 실행 중)에도 유지되는 어노테이션이다"라는 뜻
.annotation runtime Ldagger/hilt/android/HiltAndroidApp;
.end annotation


# [구역 3] 메서드 및 필드
.method public constructor <init>()V
    .locals 0
    invoke-direct {p0}, Landroid/app/Application;→<init>()V
    return-void
.end method
```

class_rename.py 고도화 진행

1. scan_and_keep_hilt_app 매서드 추가

```python
#고도화 진행(@HiltAndroidApp 어노테이션 관련)
  def scan_and_keep_hilt_app(self,smali_files: List[str]) -> List[str]:
    keep_list = []
    #HiltAndroidApp 어노테이션 문자열(smali 형식)
    target_annotation = 'Ldagger/hilt/android/HiltAndroidApp;'
    self.logger.info("Scanning for @HiltAndroidApp to apply keep rules")
    for smali_file in smali_files:
        with open(smali_file, "r", encoding="utf-8") as f:
            content=f.read()
            #파일안에 어노테이션 문자열이 있는지 확인
            if target_annotation in content:
                #정규식 클래스 이름 추출
                  class_match = util.class_pattern.search(content)
                  if class_match:
                          class_name = class_match.group("class_name")
                          keep_list.append(class_name)
                          self.logger.info(f"Keeping Hilt annotated class: {class_name}")
    return keep_list
```

class_math.group("class_name")의 결과값:
Lcom/google/samples/apps/nowinandroid/NiaApplication;

⇒이 문자열이 아래의 obfuscate 매서드의 ignore_package_names 리스트에 추가됨

⇒
파일 이름(`Lcom/.../NiaApplication;`)이 금지 목록에 들어갔으니 이름을 바꾸지 않고 건너뛰게 됨

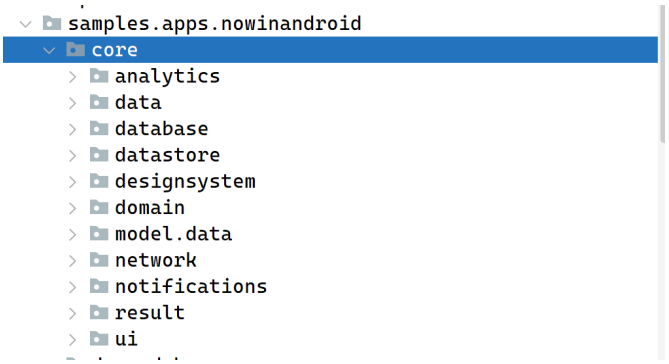
2. obfuscate 매서드 수정
   기존 ignore_package_names에 hilt_keep_classes 추가

```python
#고도화 진행(Hilt 어노테이션 관련 난독화 대상에서 제외)
hilt_keep_classes = self.scan_and_keep_hilt_app(obfuscation_info.get_smali_files())
self.ignore_package_names.extend(hilt_keep_classes)
```

패키지 단위로 고도화

네트워크,DB,로컬 저장소에 대한 위험한 구역은 통째로 class_rename 난독화 불가하게 설정

- core/network(서버 통신)
- core/database(DB)
- core/datastore(저장소)



class_rename.py 수정을 통한 고도화 진행

1. obfuscate 매서드:
   nia_ignore_package에 **core/network, core/database, core/datastroe, core/model** 를 추가 ⇒
   기존 ignore_pacakces_names리스트에 nia_ignore_package 확장
   ⇒ 플러그인 내부에서 “Now in Android” 앱의 해당 패키지를 자동으로 감지하고 보호하도록 설정

```
#고도화 진행(네트워크, DB, 데이터모델 등 난독화 대상에서 제외)
        nia_ignore_packages = [
            "Lcom/google/samples/apps/nowinandroid/core/network",   # 네트워크 (Serialization)
            "Lcom/google/samples/apps/nowinandroid/core/database",  # 로컬 DB (Room)
            "Lcom/google/samples/apps/nowinandroid/core/datastore", # 데이터 저장소 (Proto)
            "Lcom/google/samples/apps/nowinandroid/core/model",     # 데이터 모델 (직렬화 가능성)
            # 필요하다면 feature 패키지의 ViewModel도 추가 가능
        ]
        #기존 Ignore에 추가

        self.ignore_package_names.extend(nia_ignore_packages)

        self.logger.info(f"{len(nia_critical_packages)} 난독화 제외 패키지 설정됨.")
```

@AndroidEntryPoint

@AndroidEntryPoint 도 고도화 진행

비유를 하자면

- `@HiltAndroidApp` 은 본사(Applicaition)에 Hilt를 설치하는 것이라면,
  `@AndroidEntryPoint` 는 각 지점(Activity, Fragment, Service 등) 에 DI를 설치하는 것

  역할: "나(Activity/Fragment)는 Hilt한테서 의존성(객체)을 주입받을 거야!"라고 선언하는 표지판
- **대상:** `Activity`, `Fragment`, `View`, `Service`, `BroadcastReceiver` 등 안드로이드 시스템이 직접 관리하는 클래스들에 붙음

난독화 하면 안되는 이유

1. 안드로이드 시스템과의 약속
   이 어노테이션이 붙는 클래스들은 대부분 `AndroidManifest.xml` 에 등록되어 있음 ⇒ Hilt가 중간에 개입되어 있으면 클래스 이름이 바뀌어도 AndroidManifest.xml에 변경사항이 반영이 안될 수 도 있음

```
<profileable android:shell="true" />
<activity
    android:name="com.google.samples.apps.nowinandroid.MainActivity"
    android:exported="true">
    <intent-filter>
```

```java
49 import kotlinx.coroutines.BuildersKt;
50 import kotlinx.coroutines.CoroutineScope;
51 import kotlinx.coroutines.flow.FlowKt;
52 import kotlinx.coroutines.flow.StateFlow;
53 import kotlinx.datetime.TimeZone;
54 import org.jacoco.agent.rt.internal_4742761.Offline;
55 import org.jacoco.agent.rt.internal_4742761.asm.Opcodes;
56
57 /* compiled from: MainActivity.kt */
58 @Metadata(d1 = {"\u0000X\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\b\u0003\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\b\u0005\
59 @AndroidEntryPoint
60 /* loaded from: classes5.dex */
61 public final class MainActivity extends Hilt_MainActivity {
62     public static final int $stable;
63
64     @Inject
65     public AnalyticsHelper analyticsHelper;
66
67     @Inject
68     public Lazy<JankStats> lazyStats;
69
70     @Inject
71     public NetworkMonitor networkMonitor;
72
73     @Inject
74     public TimeZoneMonitor timeZoneMonitor;
75
76     @Inject
77     public UserNewsResourceRepository userNewsResourceRepository;
78
```

2. Hilt_MainActivity와의 연결
   Hilt는 `@AndroidEntryPoint` 를 보면, 컴파일 타임에 `Hilt_MainActivity` 라는 가상의 부모 클래스를 몰래 만든다
   ⇒ 클래스의 이름을 함부로 바꾸면 Hilt가 생성한 `Hilt_MainActivity` 와 연결 고리가 끊어짐
3. Reflection
   Hilt은 내부적으로 해당 액티비티나 프래그먼트를 찾아서 주입(Inject) 할 때, 클래스 이름을 참조하거나 리플랙션을 사용할 가능성이 매우 높음 but 이름이 바뀌면 주입 실패

class_rename.py 수정을 통한 고도화 진행

앞에서 @HiltAndroidApp 에 대한 처리를 확장

⇒ `target_annotations` 리스트에 `Ldagger/hilt/android/AndroidEntryPoint;` 를 추가하여, 이 문자열이 있는 파일도 난독화에서 제외

```python
#고도화 진행(@HiltAndroidApp 어노테이션 등 Hilt 관련 어노테이션을 모두 찾아 Keep 목록에 추가하는 매서드)
def scan_and_keep_hilt_app(self,smali_files: List[str]) -> List[str]:
    keep_list = []
    #찾아야 할 어노테이션들을 리스트로 정의
    target_annotations = [
        "Ldagger/hilt/android/HiltAndroidApp;", # Application용
        "Ldagger/hilt/android/AndroidEntryPoint;" # Activity/Fragment용
    ]
    self.logger.info("Scanning for @HiltAndroidApp to apply keep rules")
    for smali_file in smali_files:
        with open(smali_file, "r", encoding="utf-8") as f:
            content=f.read()
            #리스트에 있는 어노테이션 중 하나라도 있는지 확인
            #any() 함수는 리스트 중 하나라도 참이면 True
            if any(target in content for target in target_annotations):
                #정규식 클래스 이름 추출
                class_match = util.class_pattern.search(content)
                if class_match:
                    class_name = class_match.group("class_name")
                    keep_list.append(class_name)
                    self.logger.info(f"Keeping Hilt annotated class: {class_name}")
    return keep_list
```

@Inject, @HiltViewModel 추가
마찬가지로 Keep 목록에 추가하면 됨

```
#고도화 진행(@HiltAndroidApp 어노테이션 등 Hilt 관련 어노테이션을 모두 찾아 Kee
  def scan_and_keep_hilt_app(self,smali_files: List[str]) -> List[str]:
    keep_list = []
    #찾아야 할 어노테이션들을 리스트로 정의
    target_annotations = [
        "Ldagger/hilt/android/HiltAndroidApp;", # Application용
        "Ldagger/hilt/android/AndroidEntryPoint;" # Activity/Fragment용
        "Ljavax/inject/Inject;", # 생성자 주입용
        "Ldagger/hilt/android/lifecycle/HiltViewModel;" # ViewModel용

    ]
```

@Serializable 어노테이션

앞에서 네트워크,DB,로컬 패키지에 대한 고도화를 진행했지만 추가로 @Serializable 어노테이션 keep rule 규칙에 츄가해서 2차 고도화

class_rename.py 수정을 통한 고도화 진행

매서드의 이름을 scan_and_keep_hilt_app 에서scan_and_keep_critical_annotations 로 변경하고, 어노테이션 추가

```
#고도화 진행(@HiltAndroidApp 어노테이션 등  Keep 목록에 추가하는 매서드)
  def scan_and_keep_critical_app(self,smali_files: List[str]) -> List[str]:
    keep_list = []
    #찾아야 할 어노테이션들을 리스트로 정의
    target_annotations = [
        "Ldagger/hilt/android/HiltAndroidApp;", # Application용
        "Ldagger/hilt/android/AndroidEntryPoint;" # Activity/Fragment용
        "Ljavax/inject/Inject;", # 생성자 주입용
        "Ldagger/hilt/android/lifecycle/HiltViewModel;" # ViewModel용
        "Lkotlinx/serialization/Serializable;" # 직렬화용

    ]
```

obfuscate 매서드의 변수명 수정

```
#고도화 진행(Hilt 어노테이션 관련 난독화 대상에서 제외)
critical_keep_classes = self.scan_and_keep_critical_app(obfuscation_info.get_smali_files())
self.ignore_package_names.extend(critical_keep_classes)
```

@Composable

Now in Android는 Jetpack Compose 를 사용한 앱이기에 UI 코드를 보호하기 위해서 해당 어노테이션도 keep rule에 추가

class_rename.py 수정을 통한 고도화 진행

기존 코드에 아래 한 줄만 추가하면 됨

```python
#고도화 진행(@HiltAndroidApp 어노테이션 등  Keep 목록에 추가하는 매서드)
def scan_and_keep_critical_annotation(self,smali_files: List[str]) -> List[str]:
    keep_list = []
    #찾아야 할 어노테이션들을 리스트로 정의
    target_annotations = [
        "Ldagger/hilt/android/HiltAndroidApp;", # Application용
        "Ldagger/hilt/android/AndroidEntryPoint;", # Activity/Fragment용
        "Ljavax/inject/Inject;", # 생성자 주입용
        "Ldagger/hilt/android/lifecycle/HiltViewModel;", # ViewModel용
        "Lkotlinx/serialization/Serializable;",# 직렬화용
        "Landroix/compose/runtime/Composable;" # Jetpack Compose용

    ]
```

예외 처리를 통한 고도화

이전에 생성한 exceptions.py를 통해 예외처리 고도화

```python
#예외처리 고도화
from obfuscapk.exceptions import PluginExecutionError
```

```python
except Exception as e:
        # 예외 발생 시 로그
    self.logger.error(
        'Error during execution of "{0}" obfuscator: {1}'.format(
            self.__class__.__name__, e
        )
    )
    #고도화(exceptions.py를 통해 예외처리 강화)
    raise PluginExecutionError(
        plugin_name=self.__class__.__name__,
        details=str(e)
    ) from e
```

예상 결과

`ClassRename` 플러그인은 **Hilt, Data, Network, UI(Compose)** 를 모두 보호하는 아주 튼튼한 방패를 갖게 됨

난독화 진행

“Now in Android” 가 용량이 너무 커서 난독화가 잘 진행이 되지 않을 수도 있다는 발표를 듣고

Docker(WSL 2)가 사용할 수 있는 최대 메모리를 늘리는 설정 파일을 만듦

1. .wslconfig 파일 만들기

C:사용자:82102 에 만듦

- `memory=8GB` : Docker(WSL)가 최대 8GB까지 램을 쓰게 허용. (필요하면 12GB, 16GB로 늘려도 됨.)

```
[wsl2]
memory=8GB
swap=2GB
```

2. Docker 이미지 빌드(코드 수정 반영)

.wslconfig 설정을 믿고, 추가로 —memory=8g와 -Xmx6g 옵션을 넣음

```
cd "C:\Users\82102\Obfuscapk\Obfuscapk\src"

docker build -t obfuscapk .

docker run --rm -it --memory=8g -e JAVA_TOOL_OPTIONS="-Xmx6g" -v "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk:/workdir" obfuscapk --use-aapt2 -o ClassRename -o Rebuild -o NewAlignment -o NewSignature -d /workdir/result.apk /workdir/nowinandroid-debug.apk
```

진행이 잘 안됨

```
CONTAINER ID   NAME           CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O         PIDS
a85f4dfa0d3d   lucid_murdock  5.62%     43.71MiB / 8GiB     0.53%     1.7kB / 126B  82.4MB / 5.71MB   1
```

⇒

리눅스 환경으로 진입

```
PS C:\Users\82102\Obfuscapk\Obfuscapk\src> docker run -it --entrypoint /bin/bash --name fast_runner --memory=8g -e JAVA_TOOL_OPTIONS="-Xmx6g" obfuscapk
root@28144ed32817:/workdir#
```

윈도우 환경

```
PS C:\Users\82102\Obfuscapk> docker cp "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\nowinandroid-debug.apk" fast_runner:/workdir/nowinandroid.apk
Successfully copied 33.9MB to fast_runner:/workdir/nowinandroid.apk
PS C:\Users\82102\Obfuscapk>
```

리눅스 환경에서

```
root@28144ed32817:/workdir# ls -l /workdir/nowinandroid.apk
-rwxr-xr-x 1 root root 33883069 Nov  6 07:27 /workdir/nowinandroid.apk
```

```
root@28144ed32817:/workdir# python3 -m obfuscapk.cli --use-aapt2 -o ClassRename -o Rebuild -o NewAlignment -o NewSignature -d result.apk /workdir/nowinandroid.apk
```

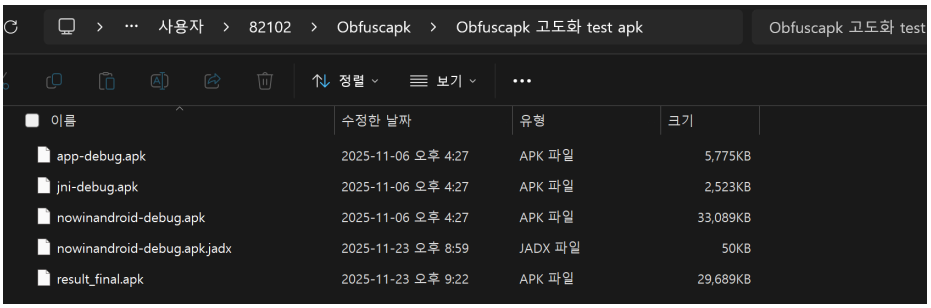
종료 후 result.apk 확인

```
root@28144ed32817:/workdir# ls -lh result.apk
-rw-r--r-- 1 root root 29M Nov 23 12:22 result.apk
```

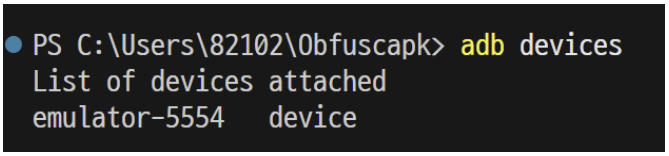
이제 result.apk를 윈도우로 가져와야 함

```
PS C:\Users\82102\Obfuscapk> docker cp fast_runner:/workdir/result.apk "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\result_final.apk"
Successfully copied 30.4MB to C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\result_final.apk
PS C:\Users\82102\Obfuscapk>
```

생성됨을 확인



애뮬레이터 돌리기(에뮬레이터 키고 vs code에서 잘 켜졌는지 확인)



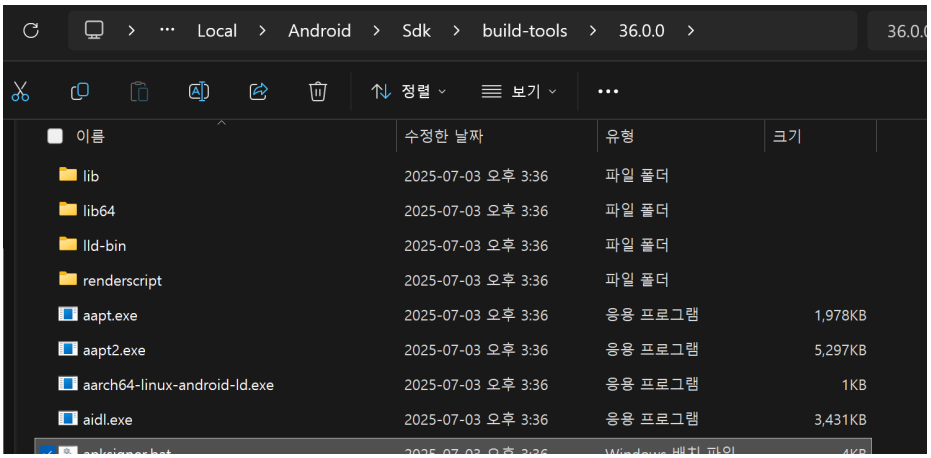
아래 명령을 통해 apk를 에뮬레이터 안에 집어넣을 때 오류 발생

```
adb install "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\result_final.apk"
```

원인:
PK 내부의 `resources.arsc` 라는 파일이 **압축되지 않은 상태(Uncompressed)로 4바이트 정렬(Aligned)** 되어 있어야 하는데, Docker 내부에서 `apktool` 이 다시 빌드(`Rebuild`)할 때 이 파일을 압축해버렸거나, `NewAlignment` 플러그인이 이를 완벽하게 펴주지 못해서 발생한 것.

⇒ 해결

1. Android SDK 도구 폴더 찾기
   C:\Users\82102\AppData\Local\Android\Sdk\build-tools
   안의 숫자 폴더 중 가장 큰 숫자 폴더로 들어가 `zipalign.exe` 와 `apksigner.bat` 가 있는지 확인



2. 수동 정렬(Zipaling)

```
# 1. 정렬 수행 (result_final.apk → result_fixed.apk 로 저장)
& "C:\Users\82102\AppData\Local\Android\Sdk\build-tools\36.0.0\zipalign.exe" -p -f -v 4 "C:\Users\82102\Obfusca
```

```
pk\Obfuscapk 고도화 test apk\result_final.apk" "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\result_fixed.apk"
```

3. 수동 서명(Apksigner)
   정렬을 다시 했기에 서명이 깨져 다시 서명

```
# 2. 서명 수행 (result_fixed.apk 에 서명)
& "C:\Users\82102\AppData\Local\Android\Sdk\build-tools\36.0.0\apksigner.bat" sign --ks "C:\Users\82102\.android\debug.keystore" --ks-pass pass:android --key-pass pass:android "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\result_fixed.apk"
```

4. 최종 설치
   result_fixed.apk를 에뮬레이터에 설치

```
PS C:\Users\82102\Obfuscapk> adb install "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\result_fixed.apk"
Performing Incremental Install
Serving...
Performing Streamed Install
Success
PS C:\Users\82102\Obfuscapk>
```

에뮬레이터를 돌려보니 바로 앱이 죽는 문제 발생

```
adb logcat -c
adb logcat *E
```

⇒ log를 찍어봄

```
PS C:\Users\82102\Obfuscapk> adb logcat -c
PS C:\Users\82102\Obfuscapk> adb logcat *:E
--------- beginning of system
11-23 12:43:10.936   716   957 E ClipboardService: Denying clipboard access to com.android.chrome, application is not in focus nor is it a system service for user 0
11-23 12:43:10.952   716   958 E ClipboardService: Denying clipboard access to com.android.chrome, application is not in focus nor is it a system service for user 0
--------- beginning of main
11-23 12:43:11.787   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=75
11-23 12:43:11.806   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=76
11-23 12:43:11.836   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=77
11-23 12:43:11.853   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=78
11-23 12:43:11.887   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=79
11-23 12:43:11.936   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=80
11-23 12:43:11.953   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=81
11-23 12:43:11.985   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=82
11-23 12:43:12.018   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=83
11-23 12:43:12.036   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=84
11-23 12:43:12.052   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=85
11-23 12:43:12.068   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=86
11-23 12:43:12.086   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=87
11-23 12:43:12.101   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=88
11-23 12:43:12.119   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=89
11-23 12:43:12.135   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=90
11-23 12:43:12.151   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
NexusLauncherActivity#324 parentId=59} producedId=14 frameNumber=117 -> producedId=14 frameNumber=91
11-23 12:43:12.169   552   552 E SurfaceFlinger: Out of order buffers detected for RequestedLayerState{com.google.android.apps.nexuslauncher/com.google.android.apps.nexuslauncher.
```

외부 라이브러리(Firebase ,DataTransport)의 내부 구조까지 건들여서 만가짐을 확인 ⇒ 라이브러리들을 keep rule에 추가

추가 고도화 진행

- `Ldagger` **(Dagger/Hilt Core):**
  - Hilt는 Dagger 위에서 돌아갑니다. `@HiltAndroidApp` 만 지킨다고 되는 게 아니라, Dagger가 내부적으로 생성한 코드(`_Factory`, `_MembersInjector`)들의 패키지 이름이 바뀌면 주입이 실패합니다.
- `Lkotlin` **(Kotlin Stdlib):**

- `IllegalAccessError` 로그 주변에 `kotlin.jvm.internal.Intrinsics` 같은 코드가 보입니다. 코틀린 내부 클래스(`functions`, `jvm` 등) 이름이 바뀌면 런타임에 치명적입니다.

- **`Landroidx/startup` & `Landroidx/lifecycle`:**
  - Now in Android는 최신 Jetpack 라이브러리를 씁니다. 이들은 앱 시작 시점에 **문자열(String)로 클래스를 찾아 초기화**하는 경우가 많습니다.

```
#고도화 진행(네트워크, DB, 데이터모델 등 난독화 대상에서 제외)
        nia_ignore_packages = [
            #앱 내부 핵심 데이터 로직
            "Lcom/google/samples/apps/nowinandroid/core/network",   # 네트워크 (Serialization)
            "Lcom/google/samples/apps/nowinandroid/core/database",  # 로컬 DB (Room)
            "Lcom/google/samples/apps/nowinandroid/core/datastore", # 데이터 저장소 (Proto)
            "Lcom/google/samples/apps/nowinandroid/core/model",     # 데이터 모델 (직렬화 가능성)
             #앱의 시작점
            "Lcom/google/samples/apps/nowinandroid/NiaApplication;",
            # 필요하다면 feature 패키지의 ViewModel도 추가 가능

            # Sync 관련 작업도 백그라운드에서 돌기 때문에 보호하는 것이 안전함
            "Lcom/google/samples/apps/nowinandroid/sync",

            # [신규 추가] 크래시를 유발하는 외부 라이브러리 방어 (Keep)
            # 1. 로그의 직접적 원인 (Firebase & Transport)
            "Lcom/google/android/datatransport",
            "Lcom/google/firebase",

            # 2. Hilt/Dagger 내부 로직 보호 (이름 바뀌면 주입 실패)
            "Ldagger",
            "Lhilt_aggregated_deps",

            # 3. Kotlin 기본 라이브러리 및 코루틴 (건드리면 런타임 오류 빈번)
            "Lkotlin",
            "Lkotlinx",

            # 4. AndroidX 핵심 컴포넌트 (Startup, Lifecycle, Arch 등 리플렉션 사용)
            "Landroidx/startup",
            "Landroidx/lifecycle",
            "Landroidx/arch",
            "Landroidx/room",  # Room 내부 클래스 보호
        ]
        #기존 Ignore에 추가
```

⇒ 다시 진행(여기 부터 다시 하기)

1단계: Docker 이미지 업데이트 (필수)

수정한 파이썬 코드를 Docker에 반영합니다.

- 위치: `C:\Users\82102\Obfuscapk\Obfuscapk\src` (터미널 경로 확인!)

PowerShell

```
docker build -t obfuscapk .
```

2단계: 고속 난독화 재실행 (Fast Method)

수정된 코드로 난독화를 다시 돌립니다. (기존 컨테이너 지우고 새로 시작)

2-1. 기존 컨테이너 삭제

PowerShell

```
docker rm -f fast_runner
```

2-2. 컨테이너 실행 & 진입

PowerShell

```
docker run -it --entrypoint /bin/bash --name fast_runner --memory=8g -e JAVA_TOOL_OPTIONS="-Xmx6g" obfuscapk
```

(이 명령어를 치면 리눅스 프롬프트 `root@...:/workdir#` 로 바뀝니다. 창 그대로 두세요.)

2-3. 새 터미널 열고 -> 파일 복사 (Windows → Docker)*(새 Windows PowerShell 창에서 실행)*

PowerShell

```
docker cp "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\nowinandroid-debug.apk" fast_runner:/workdir/nowinandroid.apk
```

2-4. 난독화 실행 (리눅스 터미널로 돌아가서 실행)

Bash

```
python3 -m obfuscapk.cli --use-aapt2 -o ClassRename -o Rebuild -o NewAlignment -o NewSignature -d result.apk /workdir/nowinandroid.apk
```

3단계: 결과물 꺼내기

난독화가 끝나면(리눅스 프롬프트가 다시 뜨면), **Windows PowerShell 창**에서 파일을 꺼냅니다.

PowerShell

```
docker cp fast_runner:/workdir/result.apk "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\result_final.apk"
```

4단계: 수동 수리 (정렬 & 서명)

새로 나온 `result_final.apk` 를 다시 수리해줍니다. (**Windows PowerShell 창**)

4-1. 정렬 (Zipalign)

PowerShell

```
& "C:\Users\82102\AppData\Local\Android\Sdk\build-tools\36.0.0\zipalign.exe" -p -f -v 4 "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\result_final.apk" "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\result_fixed.apk"
```

4-2. 서명 (Apksigner)

PowerShell

```
& "C:\Users\82102\AppData\Local\Android\Sdk\build-tools\36.0.0\apksigner.bat" sign --ks "C:\Users\82102\.android\debug.keystore" --ks-pass pass:android --key-pass pass:android "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\result_fixed.apk"
```

JAVA_HOME 환경변수 설정이 잘못되어 오류 발생

```
PS C:\Users\82102\Obfuscapk> & "C:\Users\82102\AppData\Local\Android\Sdk\build-tools\36.0.0\apksigner.bat" sign --ks "C:\Users\82102\.android\debug.keystore" --ks-pass pass:
android --key-pass pass:android "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\result_fixed.apk"

ERROR: JAVA_HOME is set to an invalid directory: C:\Users\82102\Downloads\jdk-17_windows-x64_bin.exe

Please set the JAVA_HOME variable in your environment to match the
location of your Java installation.
PS C:\Users\82102\Obfuscapk>
```

```
$env:JAVA_HOME="C:\Program Files\Android\Android Studio\jbr"
```

⇒ 다시 서명 진행

5단계: 재설치 (이제야 설치!)

이제 **새로 만든** `result_fixed.apk` 를 설치합니다.

PowerShell

```
adb uninstall com.google.samples.apps.nowinandroid
adb install "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\result_fixed.apk"
```

이 과정을 거쳐야 비로소 **"수정된 코드가 반영된 앱"**을 테스트할 수 있습니다. 번거롭더라도 꼭 1단계부터 다시 해주세요!

로그 메세지를 통한 오류 해결 및 고도화 추가 진행

다시 문제 발생

로그를 찍어보니

> PS C:\Users\82102\Obfuscapk> adb logcat *:E
> --------- beginning of system
> 11-25 07:36:22.980 736 958 E ClipboardService: Denying clipboard access to com.android.chrome, application is not in focus nor is it a system service for user 0

```
11-25 07:36:22.988   736   959 E ClipboardService: Denying clipboard access to com.android.chrome, application is
not in focus nor is it a system service for user 0
11-25 07:36:24.198   736   958 E ClipboardService: Denying clipboard access to com.android.chrome, application is
not in focus nor is it a system service for user 0
11-25 07:36:24.211   736   959 E ClipboardService: Denying clipboard access to com.android.chrome, application is
not in focus nor is it a system service for user 0
11-25 07:36:25.231   736   963 E TransitionController: Set visible without transition ActivityRecord{263082268 u0
p4d236d9a.pc822c1b6.p6ef9161b.p9a6dd283.p61d9c1a8.pfe01ce2a.pad42f669/com.google.samples.apps.nowinanc
t565} playing=false
caller=com.android.server.wm.EnsureActivitiesVisibleHelper.makeVisibleAndRestartIfNeeded:246
--------- beginning of main
11-25 07:36:26.608  8544  8544 E 1ce2a.pad42f669: hiddenapi: Accessing hidden field
Ljava/util/Collections$SynchronizedCollection;->mutex:Ljava/lang/Object; (runtime_flags=0, domain=core-platform,
api=max-target-o) from Lp363b122c$pd41d8cd9/p05c7e247/p6db9fbf6; (domain=app) using reflection: denied
11-25 07:36:26.610  8544  8544 E 1ce2a.pad42f669: hiddenapi: Accessing hidden method
Ljava/util/Collections$SynchronizedSet;-><init>(Ljava/util/Set;Ljava/lang/Object;)V (runtime_flags=0, domain=core-
platform, api=max-target-o) from Lp363b122c$pd41d8cd9/p05c7e247/p6db9fbf6; (domain=app) using reflection:
denied
11-25 07:36:26.610  8544  8544 E 1ce2a.pad42f669: hiddenapi: Accessing hidden method
Ljava/util/Collections$SynchronizedCollection;-><init>(Ljava/util/Collection;Ljava/lang/Object;)V (runtime_flags=0,
domain=core-platform, api=max-target-o) from Lp363b122c$pd41d8cd9/p05c7e247/p6db9fbf6; (domain=app)
using reflection: denied
11-25 07:36:26.786  8544  8571 E ashmem  : Pinning is deprecated since Android Q. Please use trim or other
methods.
--------- beginning of crash
11-25 07:36:26.977  8544  8544 E AndroidRuntime: FATAL EXCEPTION: main
11-25 07:36:26.977  8544  8544 E AndroidRuntime: Process:
p4d236d9a.pc822c1b6.p6ef9161b.p9a6dd283.p61d9c1a8.pfe01ce2a.pad42f669, PID: 8544

11-25 07:36:26.977  8544  8544 E AndroidRuntime: java.lang.RuntimeException: Unable to get provider
androidx.startup.InitializationProvider: androidx.startup.StartupException: java.lang.RuntimeException: Cannot find
implementation for p576f3918.p67e92c87.p86ccec3d.p73aae1a9. p73aae1a9_Impl does not exist
11-25 07:36:26.977  8544  8544 E AndroidRuntime:       at
android.app.ActivityThread.installProvider(ActivityThread.java:8647)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:       at
android.app.ActivityThread.installContentProviders(ActivityThread.java:8157)

11-25 07:36:26.977  8544  8544 E AndroidRuntime:       at
android.app.ActivityThread.handleBindApplication(ActivityThread.java:7814)

11-25 07:36:26.977  8544  8544 E AndroidRuntime:       at
android.app.ActivityThread.-$$Nest$mhandleBindApplication(Unknown Source:0)

11-25 07:36:26.977  8544  8544 E AndroidRuntime:       at
android.app.ActivityThread$H.handleMessage(ActivityThread.java:2546)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:       at android.os.Handler.dispatchMessage(Handler.java:110)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:       at android.os.Looper.loopOnce(Looper.java:248)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:       at android.os.Looper.loop(Looper.java:338)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:       at
android.app.ActivityThread.main(ActivityThread.java:9067)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:       at java.lang.reflect.Method.invoke(Native Method)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:       at
com.android.internal.os.RuntimeInit$MethodAndArgsCaller.run(RuntimeInit.java:593)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:       at
com.android.internal.os.ZygoteInit.main(ZygoteInit.java:932)
11-25 07:36:26.977  8544  8544 E AndroidRuntime: Caused by: androidx.startup.StartupException:
java.lang.RuntimeException: Cannot find implementation for p576f3918.p67e92c87.p86ccec3d.p73aae1a9.
p73aae1a9_Impl does not exist
11-25 07:36:26.977  8544  8544 E AndroidRuntime:       at
androidx.startup.AppInitializer.doInitialize(AppInitializer.java:187)
```

```
11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
androidx.startup.AppInitializer.discoverAndInitialize(AppInitializer.java:238)

11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
androidx.startup.AppInitializer.discoverAndInitialize(AppInitializer.java:206)

11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
androidx.startup.InitializationProvider.onCreate(InitializationProvider.java:45)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
android.content.ContentProvider.attachInfo(ContentProvider.java:2649)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
android.content.ContentProvider.attachInfo(ContentProvider.java:2618)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
android.app.ActivityThread.installProvider(ActivityThread.java:8642)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:        ... 11 more

11-25 07:36:26.977  8544  8544 E AndroidRuntime: Caused by: java.lang.RuntimeException: Cannot find
implementation for p576f3918.p67e92c87.p86ccec3d.p73aae1a9. p73aae1a9_Impl does not exist
11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
androidx.room.Room.getGeneratedImplementation(Room.kt:58)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
androidx.room.RoomDatabase$Builder.build(RoomDatabase.kt:1351)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
p576f3918.p67e92c87.p86ccec3d.p73aae1a9$p910eef8c.create(WorkDatabase.kt:166)

11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
p576f3918.p67e92c87.p86ccec3d.pcd679ccb.createWorkManager$default(WorkManagerImplExt.kt:34)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
p576f3918.p67e92c87.p86ccec3d.pcd679ccb.createWorkManager(Unknown Source:24)

11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
p576f3918.p67e92c87.p86ccec3d.p630c3ac7.initialize(WorkManagerImpl.java:206)

11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
p576f3918.p67e92c87.p08ff3727.initialize(WorkManager.java:210)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
p576f3918.p67e92c87.pfff77377.create(WorkManagerInitializer.java:39)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
p576f3918.p67e92c87.pfff77377.create(WorkManagerInitializer.java:30)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:        at
androidx.startup.AppInitializer.doInitialize(AppInitializer.java:180)
11-25 07:36:26.977  8544  8544 E AndroidRuntime:        ... 17 more

11-25 07:36:27.049  8544  8587 E FA      : Invalid google_app_id. Firebase Analytics disabled. See
https://goo.gl/NAOOOI. provided id: Your:App:Id
11-25 07:36:27.126  8544  8566 E FirebaseCrashlytics: Settings request failed.
11-25 07:36:27.126  8544  8566 E FirebaseCrashlytics: java.io.FileNotFoundException: https://firebase-
settings.crashlytics.com/spi/v2/platforms/android/gmp/Your:App:Id/settings?
instance=8dadf8600b595dafc0675a0aff64e281c95f355d&build_version=8&display_version=0.1.2&source=1

11-25 07:36:27.126  8544  8566 E FirebaseCrashlytics:   at
com.android.okhttp.internal.huc.HttpURLConnectionImpl.getInputStream(HttpURLConnectionImpl.java:255)
11-25 07:36:27.126  8544  8566 E FirebaseCrashlytics:   at
com.android.okhttp.internal.huc.DelegatingHttpsURLConnection.getInputStream(DelegatingHttpsURLConnection.java
11-25 07:36:27.126  8544  8566 E FirebaseCrashlytics:   at
com.android.okhttp.internal.huc.HttpsURLConnectionImpl.getInputStream(HttpsURLConnectionImpl.java:30)
11-25 07:36:27.126  8544  8566 E FirebaseCrashlytics:   at
com.google.firebase.crashlytics.internal.network.HttpGetRequest.execute(HttpGetRequest.java:82)
11-25 07:36:27.126  8544  8566 E FirebaseCrashlytics:   at
com.google.firebase.crashlytics.internal.settings.DefaultSettingsSpiCall.invoke(DefaultSettingsSpiCall.java:112)
11-25 07:36:27.126  8544  8566 E FirebaseCrashlytics:   at
com.google.firebase.crashlytics.internal.settings.SettingsController$1.then(SettingsController.java:200)
11-25 07:36:27.126  8544  8566 E FirebaseCrashlytics:   at
```

> com.google.firebase.crashlytics.internal.settings.SettingsController$1.then(SettingsController.java:193)
> 11-25 07:36:27.126 8544 8566 E FirebaseCrashlytics: at p4d236d9a.pc822c1b6.pc31b3236.pc1abb546.p2cb1ad4f.pfa646590.run(com.google.android.gms:play-services-tasks@@18.1.0:1)
> 11-25 07:36:27.126 8544 8566 E FirebaseCrashlytics: at com.google.firebase.concurrent.SequentialExecutor$QueueWorker.workOnQueue(SequentialExecutor.java:229)
> 11-25 07:36:27.126 8544 8566 E FirebaseCrashlytics: at com.google.firebase.concurrent.SequentialExecutor$QueueWorker.run(SequentialExecutor.java:174)
> 11-25 07:36:27.126 8544 8566 E FirebaseCrashlytics: at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1156)
>
> 11-25 07:36:27.126 8544 8566 E FirebaseCrashlytics: at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:651)
>
> 11-25 07:36:27.126 8544 8566 E FirebaseCrashlytics: at com.google.firebase.concurrent.CustomThreadFactory.lambda$newThread$0$com-google-firebase-concurrent-CustomThreadFactory(CustomThreadFactory.java:47)
>
> 11-25 07:36:27.126 8544 8566 E FirebaseCrashlytics: at com.google.firebase.concurrent.CustomThreadFactory$$ExternalSyntheticLambda0.run(D8$$SyntheticClass:0)
> 11-25 07:36:27.126 8544 8566 E FirebaseCrashlytics: at java.lang.Thread.run(Thread.java:1119)
> 11-25 07:36:27.175 8544 8563 E Firebase-Installations: Firebase Installations can not communicate with Firebase server APIs due to invalid configuration. Please update your Firebase initialization process and set valid Firebase options (API key, Project ID, Application ID) when initializing Firebase.
> 11-25 07:36:27.406 8544 8563 E SessionConfigFetcher: Error failing to fetch the remote configs: Bad response code: 400
> 11-25 07:36:28.717 736 1644 E TaskPersister: File error accessing recents directory (directory doesn't exist?).
> 11-25 07:36:29.729 8544 8544 E FirebaseCrashlytics: Cannot send reports. Timed out while fetching settings.
> 11-25 07:36:33.847 736 1644 E TaskPersister: File error accessing recents directory (directory doesn't exist?).
> 11-25 07:36:41.650 736 8600 E GnssPsdsDownloader: No Long-Term PSDS servers were specified in the GnssConfiguration
> 11-25 07:36:46.219 736 958 E ClipboardService: Denying clipboard access to com.android.chrome, application is not in focus nor is it a system service for user 0

- **문제:** `WorkManager`는 내부적으로 **Room 데이터베이스**를 사용하는데, 이 DB 클래스의 이름이 `p73aae1a9`로 난독화되었습니다.
- **결과:** Room은 `p73aae1a9_Impl`이라는 구현체 클래스를 찾으려다 실패하고 앱을 죽여버렸습니다.

결론: `androidx.work` (WorkManager) 패키지도 **Keep Rule(보호 목록)**에 반드시 추가해야 합니다.

```
    #  WorkManager 보호 (로그 분석 결과)
    "Landroidx/work",
]
#기존 Ignore에 추가
```

또 문제 발생

```
11-25 07:47:39.642  8806  8806 E AndroidRuntime: java.lang.IllegalAccessError: Illegal class access: 'com.google.samples.apps.nowinandroid.util.ProfileVerifierLogger' attemp
ting to access 'p4d236d9a.pc822c1b6.p6ef9161b.p9a6dd283.p61d9c1a8.p05c7e247.p994e823a$p5fb833d2$1' (declaration of 'com.google.samples.apps.nowinandroid.util.ProfileVerifier
```

로그를 분석 ⇒

- `com.google.samples.apps.nowinandroid.util.ProfileVerifierLogger` 문제가 되는 부분
- **상황:**

- `ProfileVerifierLogger` 클래스가 접근하려는 클래스들이 난독화 되어 접근 불가능한 상태가 됨

- **결론:**
  - `ProfileVerifierLogger`가 속한 패키지인 **`com.google.samples.apps.nowinandroid.util`** 패키지도 보호(Keep)해야 한다

또 문제 발생

```
11-25 08:15:33.202 10513 10513 E AndroidRuntime: java.lang.IllegalAccessError: Illegal class access: 'androidx.compose.runtime.Recomposer' attempting to access 'p576f3918.p1
8b3f3e7.pb4a61925.p3097da81$p786a2de2$1' (declaration of 'androidx.compose.runtime.Recomposer' appears in /data/app/~~W00iexsXIauFOOPMhx36Jg==/p4d236d9a.pc822c1b6.p6ef9161b.
p9a6dd283.p61d9c1a8.pfe01ce2a.pad42f669-qZhoUuoWH2raLw3R8Rbncg==/base.apk!classes12.dex)
11-25 08:15:33.202 10513 10513 E AndroidRuntime:        at androidx.compose.runtime.Recomposer.<init>(Recomposer.kt:145)
```

로그 분석 ⇒

1. **상황:** 앱이 화면(UI)을 그리려고 `Compose`를 초기화하는 중. (`WindowRecomposer` 생성)
2. **문제:** `androidx.compose.runtime` 패키지에 있는 `Recomposer` 클래스가, 내부적으로 사용하는 클래스에 접근하려고 하는데 , 그 대상이 난독화(`p576...`) 되어서 접근 권한이 막혔다.
3. **의미:** Compose 런타임(엔진)의 내부 클래스들이 서로를 못 알아보고 있다

결론: `androidx.compose` 패키지 전체를 보호(Keep)해야 한다 Compose는 코드가 서로 매우 긴밀하게 연결되어 있어서, 일부만 난독화되면 문제가 생긴다

또 문제 발생

```
11-25 08:29:15.359 10795 10795 E AndroidRuntime: java.lang.IllegalAccessError: Illegal class access: 'com.google.samples.apps.nowinandroid.core.designsystem.theme.ThemeKt' a
ttempting to access 'p4d236d9a.pc822c1b6.p6ef9161b.p9a6dd283.p61d9c1a8.pa74ad8df.pc9a8496d.pf484570d.pf500cb00$p355747b2$1' (declaration of 'com.google.samples.apps.nowinand
roid.core.designsystem.theme.ThemeKt' appears in /data/app/~~_soDJBvVw1bEZXMowDD3yg==/p4d236d9a.pc822c1b6.p6ef9161b.p9a6dd283.p61d9c1a8.pfe01ce2a.pad42f669-eWzVCxulLEim-xo56
DxyYQ==/base.apk!classes2.dex)
```

Themekt에서 문제가 발생했는데, 이는 core 안에 있고, 이전에 core/network, core/database, core/data 등을 다 보호했으니 그냥 core 패키지 자체를 보호

androidx/compose, androidx/work 이런식으로 쪼개지 말고, androidx 자체를 보호하도록 추가 설정

또 문제 발생

```
11-25 08:54:30.312 12124 12124 E AndroidRuntime: java.lang.IllegalAccessError: Illegal class access: 'com.google.samples.apps.nowinandroid.feature.foryou.ForYouViewModel' at
tempting to access 'p4d236d9a.pc822c1b6.p6ef9161b.p9a6dd283.p61d9c1a8.p518a4861.p69132687.pe2e9d4ba$pd3baf061$1' (declaration of 'com.google.samples.apps.nowinandroid.featur
e.foryou.ForYouViewModel' appears in /data/app/~~UdED9kdBhaymIsi6QkptxQ==/p4d236d9a.pc822c1b6.p6ef9161b.p9a6dd283.p61d9c1a8.pfe01ce2a.pad42f669-A_zH4oWKWBdvjvfRlRhECQ==/base
.apk!classes2.dex)
```

- **상황:** 앱이 켜지면서 홈 화면인 `For You` 탭을 띄우려고 `ForYouViewModel`을 생성
- **문제:** `ForYouViewModel` 클래스는가 난독화되어 접근 권한이 막혔습니다.
- **이유:** 우리가 `core` 패키지는 통째로 보호했지만, **`feature` 패키지는 보호하지 않았기 때문이다**
  - `ForYouViewModel`은 `feature.foryou` 패키지에 있다.
  - 이 뷰모델이 `core` 패키지에 있는 클래스(난독화 안 됨)와 상호작용할 때는 괜찮은데, 자기 자신 패키지(`feature`) 안에 있는 다른 헬퍼 클래스(난독화 됨)와 상호작용할 때 `IllegalAccessError`가 터진 것이다

  최종 해결책: `feature` 패키지도 보호하기

또 앱이 켜졌다가 죽는 문제 발생

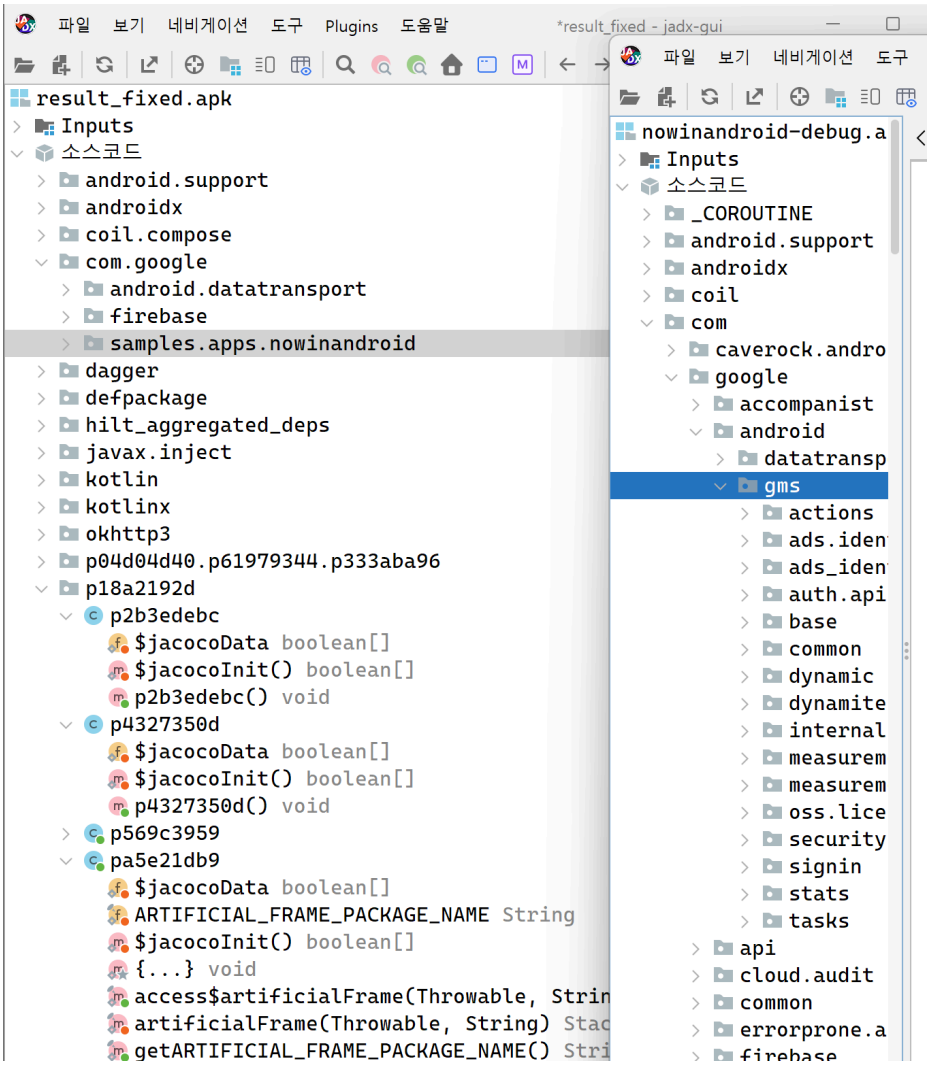
로그를 찍어보니 다른 fatal 문제는 찍히지 않음

⇒ 첫 화면이 켜지다가 꺼지는 문제이니 ui 패키지 쪽의 문제인 것 같음

⇒com.google.samples.apps.nowinandroid.ui 를 keep rule에 넣어봄

또 문제 발생

왼쪽은 난독화가 진행된 폴더, 오른쪽은 난독화가 진행되지 않은 원본 apk 파일을 jadx로 연 폴더임

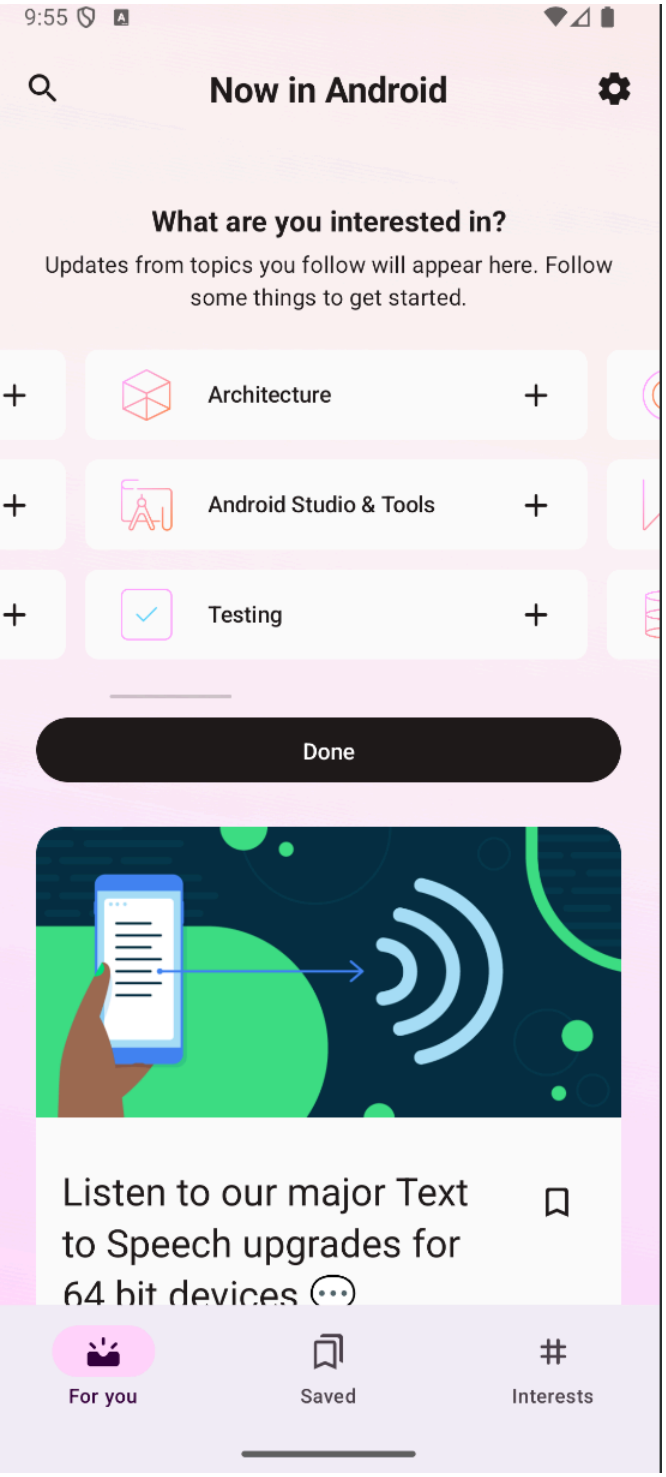


로그를 보면 com.google .android.gms 패키지에 대한 로그가 찍혀있는데 위 사진에서 왼쪽으 보면 해당 패키지 존재하지 않음 ⇒ class_rename 난독화 진행되었다는 뜻 ⇒ SecurityException: Binder invocation to an incorrect interface 에러
⇒ 외부 라이브러리를 포함하는 com.google 자체를 keep rule에 추가함

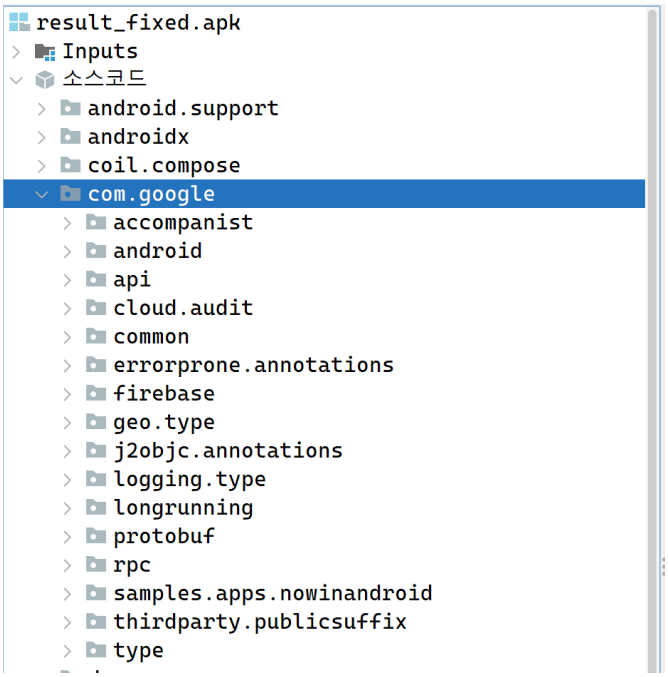
```
11-25 09:41:16.641   736  4774 E RoleControllerServiceImpl: Default/fallback role holder package doesn't qualify for the role, package: com.google.android.gms
.supervision, role: android.app.role.SYSTEM_SUPERVISION
11-25 09:41:16.645   736   765 I system_server: AssetManager2(0x787d18a79958) locale list changing from [] to [en-US]
```

성공

잘 켜지고 모든 기능이 실행됨을 확인



jadx로 확인결과 외부 라이브러리와 연결된 com.google 에 대한 keep rule이 적용되었고,

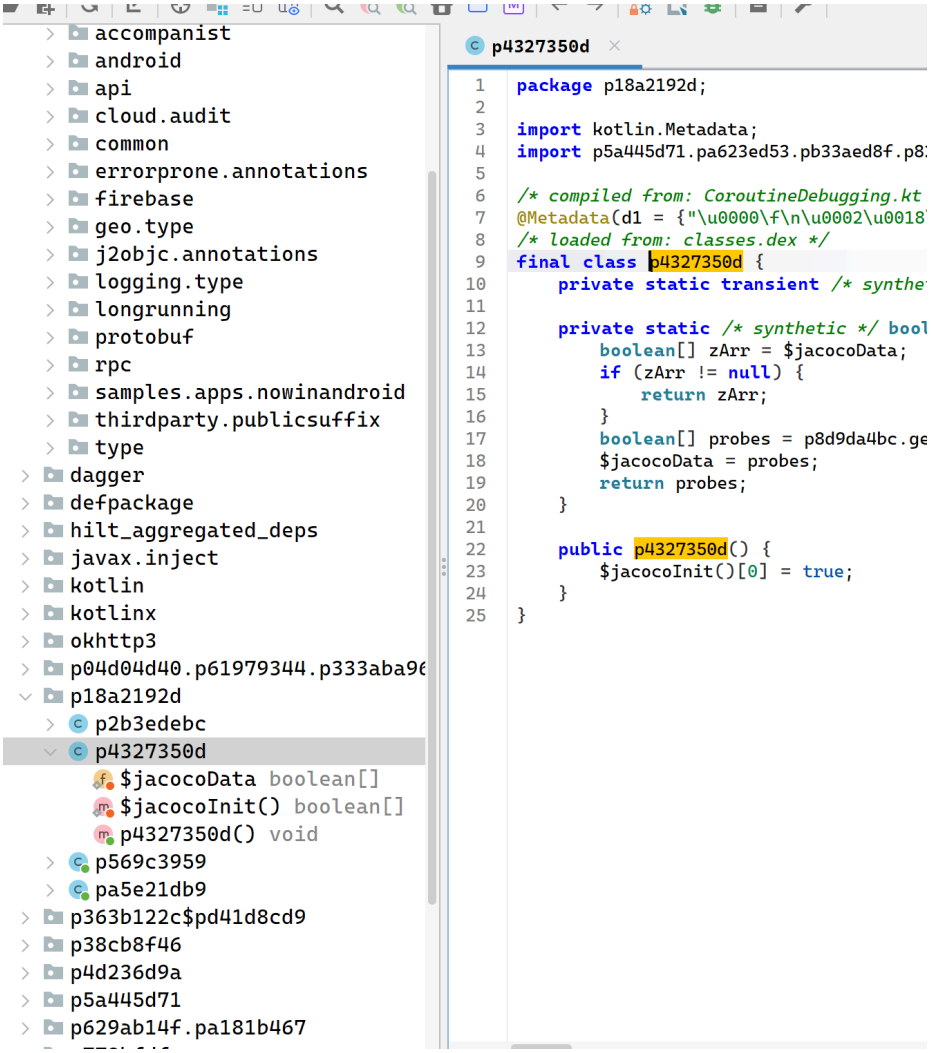


아래에 대한 keep rule들이 다 적용됨을 확인

```
nia_ignore_packages = [
        #앱 내부 핵심 데이터 로직
        "Lcom/google/samples/apps/nowinandroid/core/network",   # 네트워크 (Serialization)
        "Lcom/google/samples/apps/nowinandroid/core/database",  # 로컬 DB (Room)
        "Lcom/google/samples/apps/nowinandroid/core/datastore", # 데이터 저장소 (Proto)
        "Lcom/google/samples/apps/nowinandroid/core/model",     # 데이터 모델 (직렬화 가능성)

         #앱의 시작점
        "Lcom/google/samples/apps/nowinandroid/NiaApplication;",
        # 필요하다면 feature 패키지의 ViewModel도 추가 가능

        "Lcom/google/samples/apps/nowinandroid/core/data",
        "Lcom/google/samples/apps/nowinandroid/core/common",

        # Sync 관련 작업도 백그라운드에서 돌기 때문에 보호하는 것이 안전함
        "Lcom/google/samples/apps/nowinandroid/sync",

        #  크래시를 유발하는 외부 라이브러리 방어 (Keep)

        "Lcom/google/android/datatransport",
        "Lcom/google/firebase",
        "Lcom/google/gson",

        # 2. Hilt/Dagger 내부 로직 보호 (이름 바뀌면 주입 실패)
        "Ldagger",
        "Lhilt_aggregated_deps",
        "Lcom/google/samples/apps/nowinandroid/Hilt_", #  Hilt가 생성한 부모 클래스들
        "Lcom/google/samples/apps/nowinandroid/MainActivity", #  메인 액티비티 직접 보호 (확실하게!)

        # 3. Kotlin 기본 라이브러리 및 코루틴 (건드리면 런타임 오류 빈번)
        "Lkotlin",
        "Lkotlinx",
```

```
        # 4. AndroidX 핵심 컴포넌트 (Startup, Lifecycle, Arch 등 리플렉션 사용)
        "Landroidx/startup",
        "Landroidx/lifecycle",
        "Landroidx/arch",
        "Landroidx/room",  # Room 내부 클래스 보호

        # Compose 런타임 및 UI 보호
        "Landroidx/compose",

        #  WorkManager 보호 (로그 분석 결과)
        "Landroidx/work",

        # 유틸리티 패키지 보호
        "Lcom/google/samples/apps/nowinandroid/util",


        "Lcom/google/samples/apps/nowinandroid/core",
        "Lcom/google/samples/apps/nowinandroid/feature",

        "Lcom/google/samples/apps/nowinandroid/ui",
         "Lcom/google/samples/apps/nowinandroid/uitesthiltmanifest",
        "Lcom/google/samples/apps/nowinandroid/generated",

        "Landroidx",
        "Landroid/support",

        "Lokhttp3",
        "Lretrofit2",

        #  구글 패키지 통째로 보호 (GMS, Firebase, 앱 코드 전체 포함)
        "Lcom/google",
    ]
```

그 외의 ClassName은 난독화가 이루어 진 것을 확인함


accompanist
android
api
cloud.audit
common
errorprone.annotations
firebase
geo.type
j2objc.annotations
logging.type
longrunning
protobuf
rpc
samples.apps.nowinandroid
thirdparty.publicsuffix
type
dagger
defpackage
hilt_aggregated_deps
javax.inject
kotlin
kotlinx
okhttp3
p04d04d40.p61979344.p333aba96
p18a2192d
p2b3edebc
p4327350d
$jacocoData boolean[]
$jacocoInit() boolean[]
p4327350d() void
p569c3959
pa5e21db9
p363b122c$pd41d8cd9
p38cb8f46
p4d236d9a
p5a445d71
p629ab14f.pa181b467
p4327350d
package p18a2192d;
import kotlin.Metadata;
import p5a445d71.pa623ed53.pb33aed8f.p8
/* compiled from: CoroutineDebugging.kt
@Metadata(d1 = {"\u0000\f\n\u0002\u0018
/* loaded from: classes.dex */
final class p4327350d {
private static transient /* synthe
private static /* synthetic */ bool
boolean[] zArr = $jacocoData;
if (zArr != null) {
return zArr;
}
boolean[] probes = p8d9da4bc.ge
$jacocoData = probes;
return probes;
}
public p4327350d() {
$jacocoInit()[0] = true;
}
}